

Predicting Student Dropout: An Early-Warning Approach

Maurice Vandenheede

September 2026

1 Introduction and Motivation

Every year, some students who begin a degree do not finish it, others stay enrolled far longer than planned, and most graduate more or less on schedule. Universities know this pattern exists, but knowing which students are at risk early enough to still act on it is a different and much harder problem than describing it after the fact. This project looks at whether machine learning can help answer that question, using data from thousands of real students who went through exactly this process at a Portuguese higher education institution.

I picked this dataset partly because it looks at outcomes that matter, not an abstract benchmark. As a student myself, dropping out, staying enrolled, or graduating are not just three labels but three outcomes people around me could end up with, including me. That made this problem interesting to me.

The choice that shapes the whole project is when the prediction happens. The dataset contains a lot of information about a full year of academic performance, but a model trained on a full year of grades mostly describes what has already happened by the time the year ends. To make the prediction useful as an actual early-warning tool, one that could be used in a school, the main models in this project use only information available at enrollment and after the first semester. Semester 2 data is later added to compare with.

2 Dataset

The dataset used in this project is the UCI “Predict Students’ Dropout and Academic Success” dataset¹, collected from a Portuguese higher education institution. It contains 4424 students described by 36 features, plus a three-class target: Dropout, Enrolled, or Graduate.

The features fall into three groups. Twenty-four are known at the moment a student enrolls: demographics such as age and marital status, economic and social background, and details of the application itself. Six describe academic performance at the end of the first semester, units enrolled, evaluated, approved, and average grade among them, and six more describe the same at the end of the second semester.

Combining the enrollment-time features with the first semester gives the 30-feature early-warning set used for the main models in this project. The six second-semester features are kept and used only in the separate comparison in Section 6. The dataset has no missing values in any column.

The target classes are imbalanced. Graduate accounts for 49.9 percent of students, Dropout for 32.1 percent, and Enrolled for 17.9 percent, as shown in Figure 1. Enrolled is both the smallest class and, as the results in Section 5 show, the hardest one to predict.

¹Realinho, V., Vieira Martins, M., Machado, J., and Baptista, L. (2021). Predict Students’ Dropout and Academic Success. UCI Machine Learning Repository. <https://doi.org/10.24432/C5MC89>

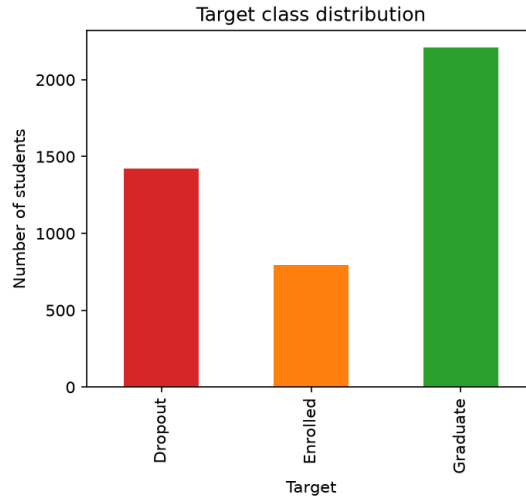


Figure 1: Distribution of the target classes across the 4424 students in the dataset.

3 Hypotheses

Before running any models, I wrote down what I actually expected to happen. Stating this upfront, rather than only after seeing the results, makes it possible to say honestly which expectations held up and which did not.

- **H1:** Tree-based ensembles (Random Forest, Gradient Boosting) will outperform Logistic Regression, since dropout is likely to involve non-linear relationships and interactions between features that a linear model cannot capture.
- **H2:** Balancing the classes during training will trade some overall accuracy for better recall on the minority Enrolled class, rather than improving both at once.
- **H3:** Adding second semester features will improve accuracy over the early-warning set, at the cost of how early the prediction can actually be made.
- **H4:** Random Forest’s random feature subsampling will outperform plain Bagging, since decorrelating the trees should reduce variance more than using every feature at each split.

4 Methodology

4.1 Pipeline and Evaluation Strategy

Every model in this project is built as a single scikit-learn Pipeline, combining preprocessing with the model itself rather than treating preprocessing as a separate step done once upfront. Numeric features are standardized with StandardScaler, and categorical features are one-hot encoded with OneHotEncoder. Wrapping both together in a Pipeline means the preprocessing is always fit only on the training data for each split, then applied to whatever is held out, whether that is the test set or a single cross-validation fold. Without this, information from the data being evaluated could leak into how the training data is scaled or encoded, which would make the reported performance look better than it really is.

The full dataset is split once into a training set and a test set, 80 percent training and 20 percent testing, stratified so the class proportions are preserved in both. A fixed random seed (`random_state=42`) is reused for this split and everywhere else randomness enters the pipeline, so every model in this report is evaluated on exactly the same held-out rows.

All hyperparameter tuning uses 5-fold cross-validation on the training split only. The scoring metric is balanced accuracy rather than plain accuracy, a deliberate choice given how imbalanced the target classes are: with plain accuracy, a model could score well just by favoring the majority Graduate class and largely ignoring Enrolled. Balanced accuracy averages the recall obtained on each class, so all three classes count equally regardless of how many students belong to each one. The test set is used exactly once per model, for the final numbers reported in Section 5, never during tuning. This is what makes the reported test accuracy an honest estimate rather than a number quietly fit to the test set.

4.2 Models

The five models below are presented in the order the course covered them, since each builds on a limitation of the one before it: from a purely linear baseline, up through trees that correct each other’s mistakes.

Logistic Regression. This is the linear baseline. With three target classes, scikit-learn’s solvers switch automatically to a multinomial, or softmax, formulation. Both L1 and L2 regularization were tuned via 5-fold cross-validation; the two reached almost the same score, and L1 with $C=1$ was kept for its sparser, more interpretable coefficients (Section 7).

Decision Tree. A single tree comes next, included to show a problem the ensembles below are built to solve: high variance. An unpruned tree can reach near-perfect training accuracy while performing far worse on new data. Cost-complexity pruning controls this, and `ccp_alpha` was tuned using candidates from the tree’s own pruning path rather than a guessed range. Figure 2 shows the effect: training accuracy stays high while cross-validation accuracy peaks around `ccp_alpha=0.0008` then declines. The best value found was `ccp_alpha=0.00078`.

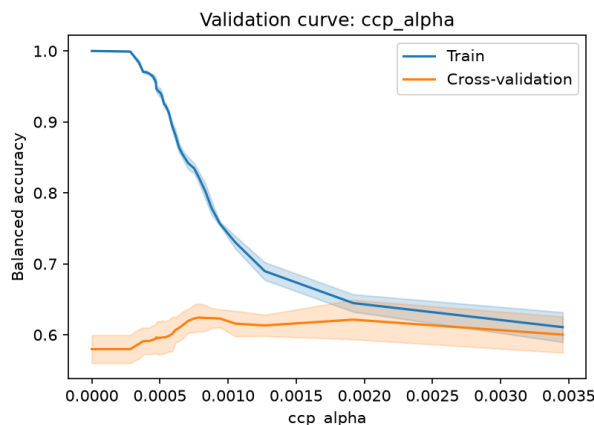


Figure 2: Training versus 5-fold cross-validation balanced accuracy across the Decision Tree’s cost-complexity pruning path. The gap between the two curves at low `ccp_alpha` is the overfitting the pruned models later in this section are designed to avoid.

Bagging. Bagging fits many trees to different bootstrap samples and averages their predictions, reducing the variance problem above without pruning. In scikit-learn this is a Random Forest with `max_features=None`, using every feature at every split, exactly what bagging is: m equals p . It was left untuned, serving as a clean $m=p$ reference point for the tuned Random Forest below.

Random Forest. Random Forest extends bagging by restricting each split to a random subset of features, so m is smaller than p , which decorrelates the trees. Rather than the usual rule of thumb, m equal to the square root of p , roughly 11 of the 119 one-hot encoded columns here, m was tuned via cross-validation. The search found $m=46$ performed best, about four times higher than that rule of thumb. Tree count was checked separately for a plateau, since more trees cannot cause a bagging-based method to overfit, and 300 trees were kept once the curve flattened.

Gradient Boosting. Gradient Boosting also combines many trees, but fits them sequentially: each new tree targets the residual error left by the trees fit so far, rather than averaging independently trained trees. Depth, learning rate, and number of trees were tuned in that order, each held fixed before the next, faster than a joint search but with a risk of missing interactions (Section 9). The final configuration used a max depth of 5, a learning rate of 0.1, and 200 trees.

4.3 Handling Class Imbalance

To directly address the class imbalance beyond what balanced accuracy already does during tuning, most models were also trained in a class-imbalance-aware variant alongside their baseline. For Logistic Regression, Decision Tree, and Random Forest, this uses scikit-learn’s `class_weight="balanced"` option, which reweights the loss so that a misclassified Enrolled student, the minority class, counts several times more than a misclassified Graduate student, roughly in inverse proportion to how often each class appears in the training data. Gradient Boosting has no `class_weight` parameter, so the same inverse-frequency weighting is applied through `sample_weight` at fit time instead. Bagging is not tuned, as noted above, but a balanced variant was trained for it too, using the same `class_weight` mechanism as Random Forest, so it remains comparable to the other four models.

5 Results

5.1 Overall Performance

Table 1 lists test accuracy and Enrolled-class precision and recall for all ten model and variant combinations, evaluated on the early-warning feature set. The best single result belongs to the baseline Logistic Regression, at 0.748, the simplest model tried rather than the most complex one.

Model	Variant	Test accuracy	Enrolled precision / recall
Logistic Regression	baseline	0.748	0.50 / 0.33
Logistic Regression	balanced	0.695	0.37 / 0.57
Decision Tree	baseline	0.715	0.43 / 0.32
Decision Tree	balanced	0.638	0.30 / 0.48
Bagging	baseline	0.737	0.47 / 0.31
Bagging	balanced	0.713	0.40 / 0.48
Random Forest	baseline	0.729	0.44 / 0.28
Random Forest	balanced	0.720	0.42 / 0.49
Gradient Boosting	baseline	0.724	0.41 / 0.34
Gradient Boosting	balanced	0.712	0.39 / 0.49

Table 1: Test accuracy and Enrolled-class precision/recall for all ten model and variant combinations, early-warning feature set.

Excluding one case, every model and variant lands between 0.69 and 0.75, a narrow spread given how different the five algorithms are. The exception is the balanced Decision Tree, at 0.638, noticeably below every other result in the table. Applying class weights to an already pruned, fairly shallow tree seems to cost it more than it costs any of the ensemble methods, a point worth returning to in the discussion.

Figure 3 compares each model’s 5-fold cross-validation accuracy against its held-out test accuracy. The two track each other closely, generally within one or two accuracy points and never more than about three, which is itself a useful check: if the tuning process had overfit to the cross-validation folds, test accuracy would be expected to fall well short of the cross-validated estimate, and it does not. For context, other published work on this same dataset reports 83 to 86 percent accuracy, a comparison addressed directly in Section 8.

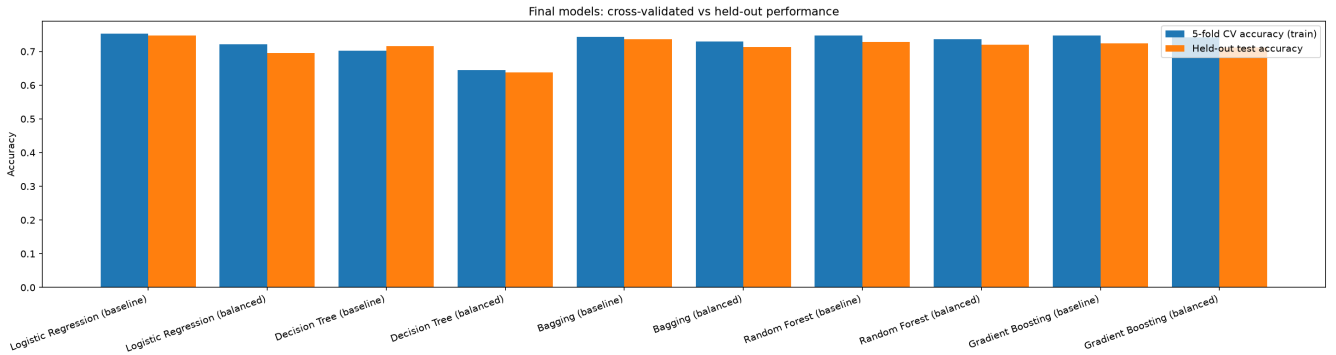


Figure 3: 5-fold cross-validation accuracy versus held-out test accuracy for all ten model and variant combinations.

5.2 Per-Class Performance

Overall accuracy hides how unevenly each model performs across the three classes. Figure 4 breaks recall down by class for all ten combinations. Graduate is reliably well identified, recall between 0.71 and 0.90 depending on the model, and Dropout sits in the middle, roughly 0.61 to 0.74. Enrolled is a different story: recall ranges from 0.28 to 0.57, always the lowest of the three classes, and this is the number that actually matters for an early-warning system, since Enrolled is the group a study advisor would most want flagged.

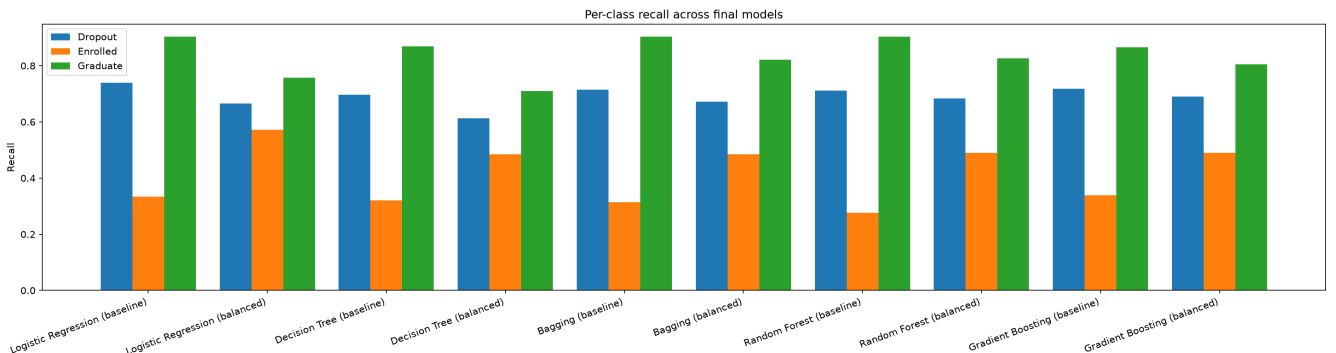


Figure 4: Per-class recall for all ten model and variant combinations. Enrolled is consistently the hardest class to identify.

Figure 5 shows this concretely for Logistic Regression. The baseline model correctly identifies only 53 of 159 Enrolled students, and misclassifies more of the remaining 106 as Graduate (71) than as Dropout (35), so the model’s main error is treating at-risk enrolled students as if they

were on track to graduate. The balanced variant raises this to 91 of 159 correctly identified, at a direct cost elsewhere: correctly identified Graduates drop from 399 to 335, with 83 actual Graduates now predicted as Enrolled instead of only 22 before. Balancing does not create better information, it only shifts where the model’s mistakes fall, and whether that shift is worth it depends on which mistake costs more in practice, a question taken up directly in Section 8.

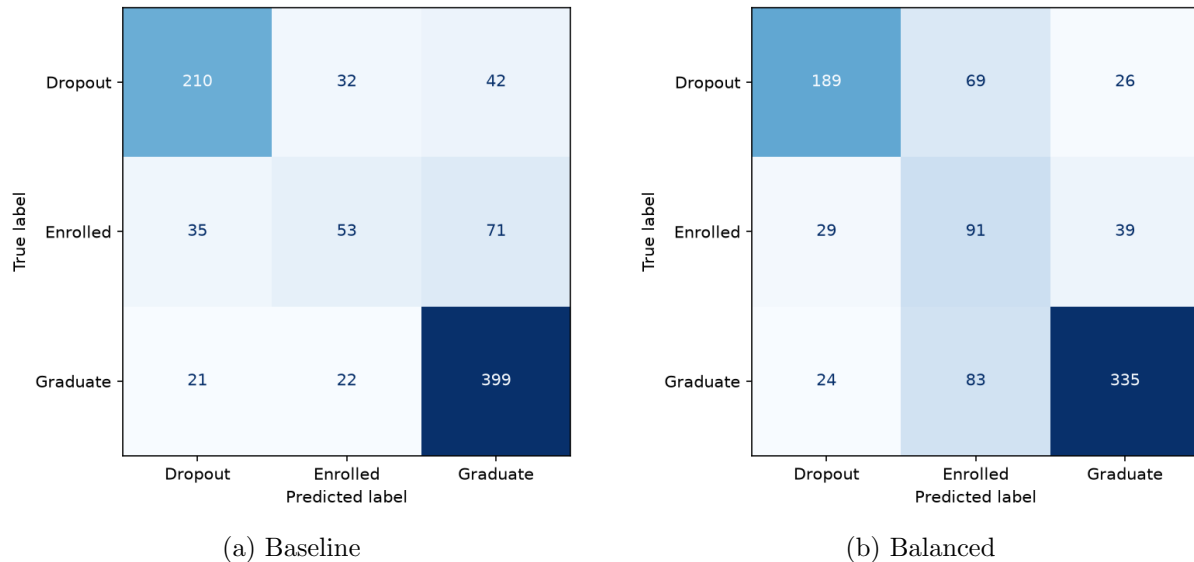


Figure 5: Logistic Regression confusion matrices, baseline versus balanced, on the held-out test set (885 students).

6 Early-Warning vs. Full-Year Comparison

The early-warning framing is a real restriction: leaving out the six second-semester features means giving up information that is available by the end of the year and clearly relevant to the outcome. This section quantifies what that restriction actually costs in accuracy.

Each model’s baseline variant was retrained using the same tuned hyperparameters found in Section 4, changing only the feature set from the 30-feature early-warning set to the full 36-feature set. Reusing the hyperparameters rather than retuning them separately for each feature set is a deliberate choice: it isolates the effect of the extra features from any change in model complexity, so the comparison is fair even though it means the full-year models here are not necessarily the best full-year models one could build.

Every model improves with the extra features, but not by much, as Figure 6 shows: Decision Tree gains only 0.008, Logistic Regression, Bagging, and Gradient Boosting each gain around 0.025, and Random Forest gains the most, 0.049, moving from 0.729 to 0.777. The size of the gain seems related to how well a model can already exploit the correlation between semester 1 and semester 2 performance from the early-warning features alone: Random Forest, which benefits most from additional structure to split on, gains the most, while the single Decision Tree, already constrained by pruning to a fairly small number of splits, gains almost nothing.

A one to five point gain is a modest price for keeping the prediction early enough to be useful, and it is far smaller than the gap to the 83 to 86 percent accuracy reported in other published work on this dataset. That gap, and why it is mostly not about which semester of data is used, is addressed in Section 8.

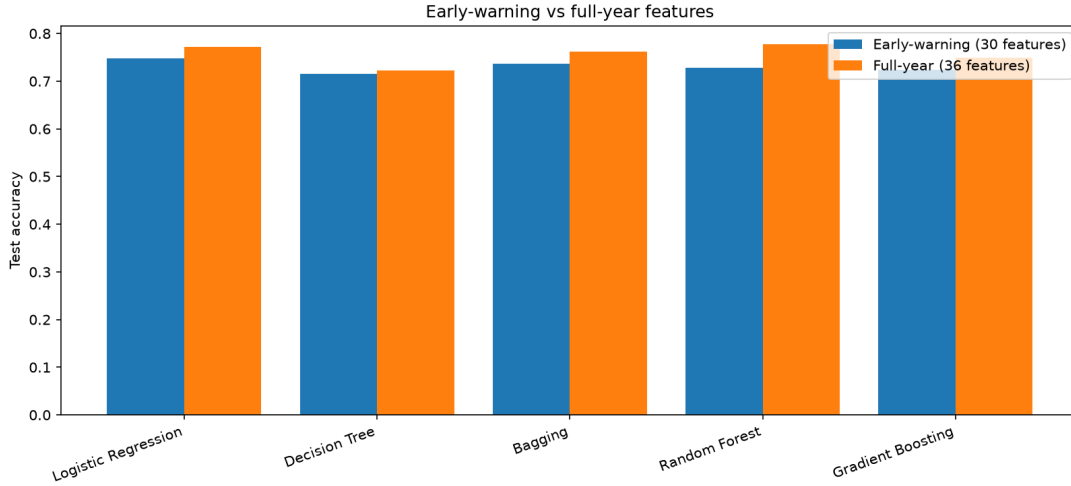


Figure 6: Test accuracy for each model’s baseline variant, early-warning feature set versus full-year feature set, using the same tuned hyperparameters for both.

7 Feature Interpretation

One reason the L1-regularized Logistic Regression from Section 4.2 is worth a closer look, beyond being the best-scoring model in Table 1, is that its coefficients can be read directly. A positive coefficient for a class pushes the predicted probability of that class up, holding the other features fixed, and a negative one pushes it down. Figure 7 shows the eleven largest coefficients for each of the three classes.

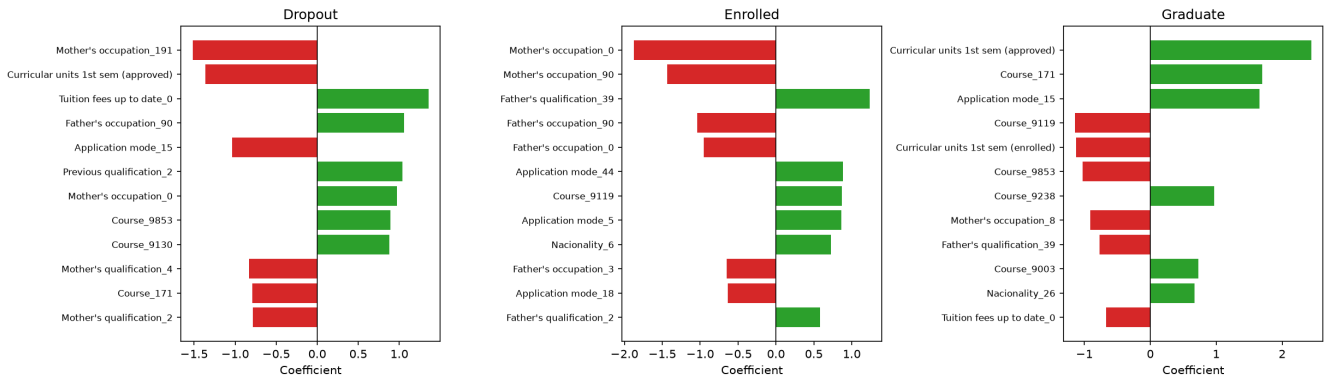


Figure 7: Largest Logistic Regression coefficients per class. Green bars push the prediction toward that class, red bars push away from it.

Curricular units 1st sem (approved), how many first-semester courses a student actually passed, is the clearest signal in the model. It is by far the largest coefficient for Graduate, and among the largest for Dropout, with the expected sign in both cases: approving more units pushes the prediction toward Graduate, approving fewer pushes it toward Dropout. This lines up with Section 5, where semester 1 performance already carries most of what the early-warning feature set has to offer.

A second interpretable result is the dummy variable for students whose tuition is not up to date, which carries one of the largest positive coefficients for Dropout. Financial standing shows up directly in the model, despite most of the remaining categorical features being encoded as anonymous UCI codes rather than readable labels.

The Enrolled panel looks different from the other two. None of its largest coefficients belong to a numeric feature like curricular units, every one is a category code for something like parents’

occupation, qualification, application mode, or course. There is no single dominant signal the way there is for Dropout and Graduate, which may be part of why Enrolled is consistently the hardest class to predict, as seen earlier in Section 5.2: the model has less to clearly latch onto.

Most of the categorical coefficients are labeled with raw UCI codes, for example `Course_9853` or `Mother's occupation_191`, rather than the category names they represent. Mapping these back to readable labels using the UCI documentation would make this section more directly useful, and is listed as future work in Section 9.

8 Discussion

The headline result, roughly 73 percent accuracy on the early-warning set, is a genuine improvement over both naive baselines: 50 percent for always guessing Graduate, and 33 percent for random three-class guessing. More telling than the number itself is how consistent it is: five very different algorithms, a linear model, a single tree, two ensembles, and a boosted model, all land within about 0.69 to 0.75, excluding the class-weighted Decision Tree discussed below. That consistency points to a ceiling set by what the 30 available features can explain, not by how hard any model tried, and it leaves H1 unconfirmed: the single best result is the plain Logistic Regression baseline, the simplest model tried.

Part of that ceiling is almost certainly the dataset itself: it captures academic and demographic detail but nothing about motivation, financial stability, or personal circumstances, exactly the kind of factors that matter for a decision this personal. No model can recover information that was never collected.

The same ceiling explains most of the gap to other published work on this dataset, which reports 83 to 86 percent accuracy using the full 36 features and SMOTE oversampling instead of class weights. Section 6 already showed that semester 2 features add only one to five points here, far short of a ten-plus point gap, so oversampling looks like the larger factor, and that work is really answering an easier question: predicting the final outcome with a full year of data, not flagging risk early enough to act on.

H4 also did not hold up: Bagging's baseline (0.737) beat Random Forest's tuned baseline (0.729), even though random feature subsampling is supposed to decorrelate the trees and help. The cross-validated search settled on $m=46$, well above the rule-of-thumb value of roughly 11, suggesting the features here are different enough from each other that restricting splits helps less than the standard argument assumes.

Accuracy is also the wrong number to optimize for this problem. Enrolled recall, 0.28 to 0.57 across the ten combinations, is what actually matters for flagging at-risk students, since Graduate is already reliably identified around 0.90 recall regardless of variant. A false positive costs an unnecessary conversation; a false negative costs the one thing the tool exists to prevent. That asymmetry justifies treating the balanced variants as a real design choice, confirming H2: balancing always traded accuracy for Enrolled recall, never improving both at once.

The one unusually expensive trade is the balanced Decision Tree, at 0.638 (Section 5.1), likely because a single pruned tree has only a small, fixed number of leaf regions to redraw, while the ensembles spread the same reweighting across hundreds of trees and Logistic Regression adjusts a smooth, continuous boundary, both with more room to absorb the change.

9 Limitations and Future Work

- **Neural networks.** ISLP covers the basics of neural networks (10.1, 10.2, 10.7), but they were not tried here. With roughly 3500 training rows and 30 features, a neural network would likely struggle to outperform the linear model and tree ensembles already tested, since neural networks tend to need considerably more data before their extra capacity pays

off. This is an assumption, not something tested directly, so it remains a natural next step rather than a settled conclusion.

- **A joint hyperparameter search for Gradient Boosting.** Section 4.2 tuned max depth, learning rate, and number of trees one at a time, in that order, which is faster than a full joint search but can miss combinations where two hyperparameters interact. A joint search, a grid or randomized search across all three at once, is the natural way to check whether the sequential approach left performance on the table.
- **SMOTE or another oversampling technique.** Section 8 discussed SMOTE as an alternative way of handling the class imbalance, used in other published work on this dataset but not implemented anywhere in this project. Trying it here, instead of or alongside class weighting, would directly test whether it explains part of the gap to that published work, rather than only inferring it from the comparison.
- **Richer categorical decoding.** The one-hot encoded categorical features in Figure 7 appear as raw UCI codes, for example `Course_9853`, rather than the category they represent. Mapping these back to readable labels using the UCI documentation would make the coefficient and feature importance plots more directly useful, without changing anything about the models themselves.
- **Features not present in this dataset.** A student's motivation, day-to-day financial situation, or personal circumstances are plausibly relevant to dropout and simply were not collected here. This is likely part of the accuracy ceiling discussed in Section 8, and no amount of additional modeling on the current features can recover it.

10 Conclusion

This project set out to answer a specific question: can a model trained only on enrollment-time and first-semester information flag students at risk of dropping out early enough to still act on it. The answer is a qualified yes. Every model beat both naive baselines by a wide margin, and the best result, a plain Logistic Regression at 0.748 accuracy, needed nothing more sophisticated than the course's own linear baseline to get there. That was the most useful lesson of the project: a more complex model is not automatically a better one, and testing that assumption directly mattered more than any single hyperparameter choice.

The real limitation is not any model tried but what the data can support. Enrolled recall stayed the hardest number to move, and no amount of tuning changed that as much as simply deciding whether to trade accuracy for recall on that one class did. Restricting the feature set to keep the prediction early cost one to five accuracy points, a reasonable price for a prediction that could still be acted on instead of arriving too late to matter.

Working through this dataset, from an early attempt at the tree-based models that leaned on unfamiliar shorthand before the underlying ideas were solid, to a full early-warning pipeline with ten evaluated model variants, was where the theory from the exam turned into something I actually understand rather than something I only remembered for the exam.